

GRAVITY — Full System Architecture Document

Product: Gravity Family Safety Platform
Version: 1.0
Date: 19 June 2026
Repository: github.com/kamaralam1984/gravitypro
Domain: gravity.trackalways.com

Table of Contents

- 1. [Executive Summary](#)
- 2. [Technology Stack](#)
- 3. [System Architecture Overview](#)
- 4. [Infrastructure & Deployment](#)
- 5. [Backend API Architecture](#)
- 6. [Database Schema](#)
- 7. [Database Indexes & Performance](#)
- 8. [Authentication System](#)
- 9. [Real-Time Communication \(SSE\)](#)
- 10. [File Storage — Cloudflare R2](#)
- 11. [Web Application Architecture](#)
- 12. [Mobile Application Architecture](#)
- 13. [Mobile App Permissions](#)
- 14. [Push Notifications](#)
- 15. [Geofencing System](#)
- 16. [Security Model](#)
- 17. [API Endpoint Reference](#)
- 18. [API Error Response Format](#)
- 19. [API Request & Response Examples](#)
- 20. [Data Flow Diagrams](#)
- 21. [Third-Party Service Dependencies](#)
- 22. [Scalability & Known Limitations](#)
- 23. [Monitoring & Health Checks](#)
- 24. [Backup & Disaster Recovery](#)
- 25. [Data Privacy & Compliance](#)
- 26. [Target Markets](#)
- 27. [Development Setup Guide](#)
- 28. [Environment Configuration](#)
- 29. [Project File Structure](#)

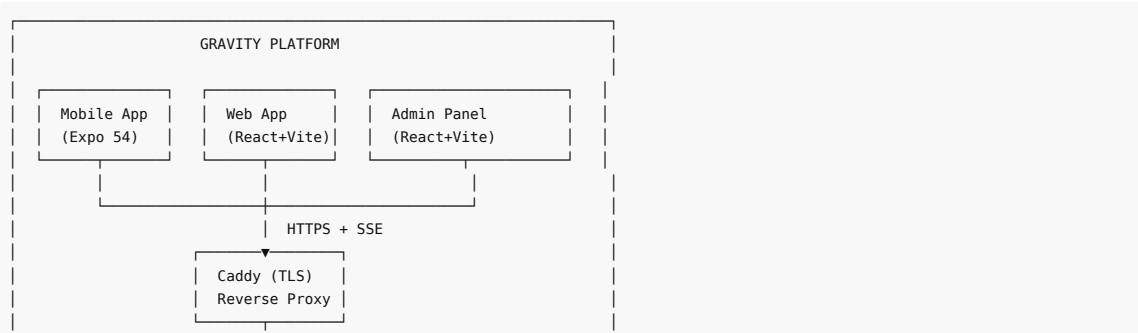
1. Executive Summary

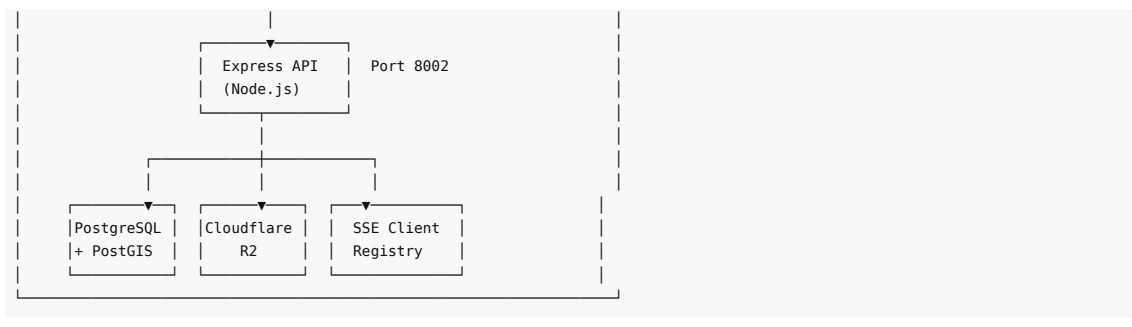
Gravity is a **family safety** and **real-time location tracking platform** designed to help families stay connected and safe. The platform enables parents to monitor the live location of family members, define safe zones (geofences), receive instant SOS alerts, and manage all family groups through a premium web interface and mobile app.

Core Capabilities

Capability	Description
Live Location Tracking	Real-time GPS coordinates broadcast to all family members via Server-Sent Events
SOS Alerts	One-tap emergency button that instantly notifies all family members via push notification + in-app alert
Geofencing	PostGIS-powered geographic zones — automatic entry/exit event logging
Family Circles	Invite-code-based family groups; multiple circles per user supported
Admin Control	Full administrative panel with user management, system monitoring, and broadcast messaging

Platform Components





2. Technology Stack

Backend

Technology	Version	Purpose
Node.js	20.x LTS	Runtime environment
Express.js	5.2.x	HTTP server framework
PostgreSQL	15+	Primary relational database
PostGIS	3.x	Geospatial extension for safe zone geometry
JWT	jsonwebtoken	Stateless authentication tokens
bcrypt	bcryptjs	Password hashing
Zod	3.x	Runtime request schema validation
express-rate-limit	7.x	API rate limiting
CORS	cors	Cross-origin resource sharing
dotenv	16.x	Environment variable management

Web Frontend

Technology	Version	Purpose
React	18.x	UI library
TypeScript	5.x	Type safety
Vite	5.x	Build tool and dev server
CSS Modules	—	Scoped component styles
Leaflet.js	1.9.x	Interactive maps (4 tile styles)
react-leaflet	4.x	React bindings for Leaflet

Mobile Application

Technology	Version	Purpose
React Native	0.74+	Cross-platform mobile framework
Expo	54	Development platform and managed workflow
expo-location	17.x	Device GPS access
expo-notifications	0.28.x	Push notification handling
expo-blur	13.x	iOS-style blur effects
react-native-maps	1.14.x	Native map integration (Google Maps / Apple Maps)
@react-navigation/native	6.x	Screen navigation
@react-navigation/bottom-tabs	6.x	Bottom tab navigation
react-native-safe-area-context	4.x	Safe area insets for notched devices
@expo/vector-icons	14.x	Ionicons icon set

Infrastructure

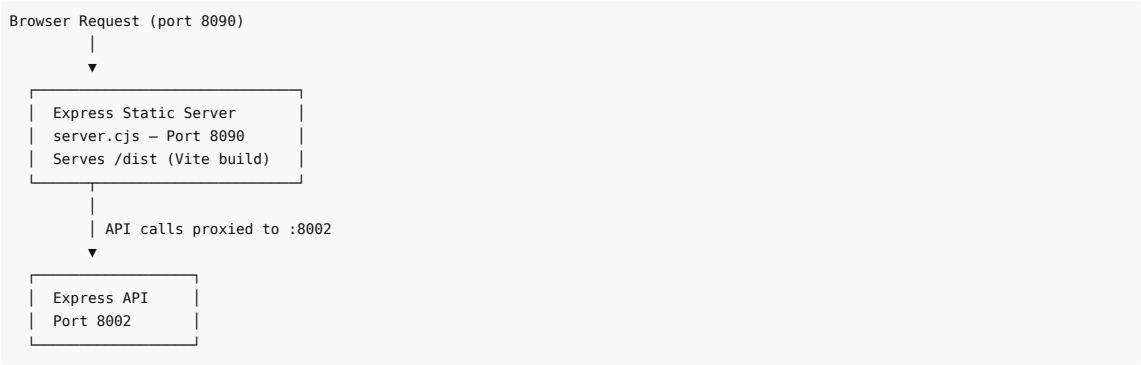
Technology	Purpose
Caddy 2	Reverse proxy with automatic TLS (Let's Encrypt)
PM2	Node.js process manager with auto-restart
Cloudflare R2	S3-compatible object storage for user avatars
MSG91	SMS OTP delivery service
Expo Push API	Mobile push notification delivery

3. System Architecture Overview

Request Flow



Web Frontend Serving



4. Infrastructure & Deployment

Process Management — PM2

Three processes managed by PM2 via `ecosystem.config.js` :

Process Name	Details
gravity-api	Express backend – Port 8002 Max memory: 500MB, autorestart: true
gravity-web	Static file server – Port 8090 Max memory: 200MB, autorestart: true
gravity-traccar	Traccar GPS ingestion service Port 8082 (Caddy routes <code>/telemetry/</code>)

Commands:

```
pm2 start ecosystem.config.js # Start all processes
pm2 reload ecosystem.config.js # Zero-downtime reload
pm2 logs gravity-api          # View API logs
pm2 monit                     # Real-time dashboard
```

Caddy Reverse Proxy

Caddy handles TLS termination and routing at `gravity.trackalways.com` :

```
Request → gravity.trackalways.com
|
├─ /api/v1/sse/* → localhost:3000 (flush_interval: -1 for SSE streaming)
├─ /telemetry/* → localhost:8082 (Traccar GPS ingestion)
└─ /api/*       → localhost:3000 (Express API)
```

SSE Special Config: `flush_interval -1` disables response buffering — critical for Server-Sent Events to stream in real-time without Caddy buffering event chunks.

Security Headers Applied by Caddy:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Referrer-Policy: strict-origin-when-cross-origin`
- gzip compression on all responses

Logs

Process	Output Log	Error Log
gravity-api	<code>/tmp/gravity-api-out.log</code>	<code>/tmp/gravity-api-error.log</code>
gravity-web	<code>/tmp/gravity-web-out.log</code>	<code>/tmp/gravity-web-error.log</code>
gravity-traccar	<code>/tmp/gravity-traccar-out.log</code>	<code>/tmp/gravity-traccar-error.log</code>
Caddy	<code>/var/log/caddy/gravity.log (JSON)</code>	—

5. Backend API Architecture

App Entry Point — `backend/src/app.js`

```
app.js
|
├─ Global middleware
|   └─ cors({ origin: '*' })
|   └─ express.json()
|   └─ express-rate-limit (1000 requests / 15 minutes per IP)
|
├─ Route mounting
|   └─ /api/v1/auth → routes/auth.js
|   └─ /api/v1/users → routes/users.js
|   └─ /api/v1/circles → routes/circles.js
|   └─ /api/v1/sos → routes/sos.js
|   └─ /api/v1/geofences → routes/geofences.js
|   └─ /api/v1/locations → routes/locations.js
|   └─ /api/v1/media → routes/media.js
```

```

├── /api/v1/sse      → routes/sse.js
├── /api/v1/admin    → routes/admin.js
├──
└── Global error handler
    └── Returns { error: message } JSON on uncaught route errors

```

Middleware

authenticate — backend/src/middleware/auth.js

- Reads Authorization: Bearer <token> header
- Verifies JWT with JWT_SECRET from environment
- Attaches req.user = { id, phone, role } to request
- Returns 401 Unauthorized if token missing or invalid

validate(schema) — backend/src/middleware/validate.js

- Accepts a Zod schema object
- Parses req.body against schema
- Returns 400 Bad Request with field-level error details on mismatch
- On success, replaces req.body with the parsed (typed) result

adminAuth — inline in routes/admin.js

- Reads x-admin-token request header
- Compares against ADMIN_TOKEN environment variable
- Returns 401 Unauthorized on mismatch

Database Connection — backend/src/config/db.js

Uses node-postgres (pg) with connection pooling:

```

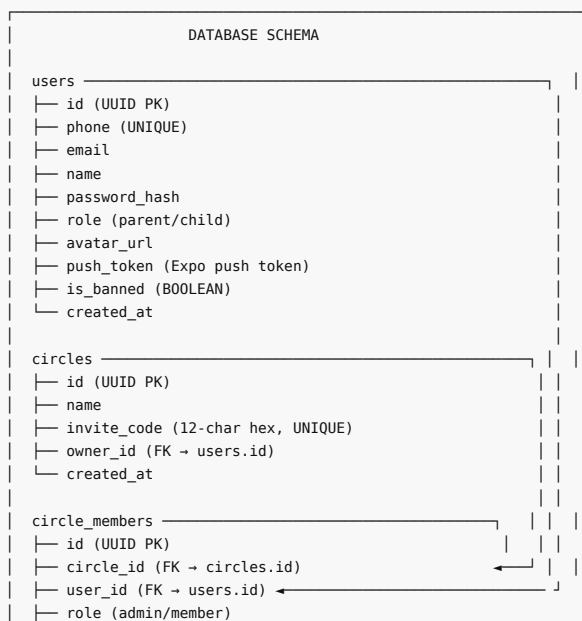
// Pool configuration
const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  max: 20,           // max pool size
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
})

// All queries use parameterized statements ($1, $2...)
// No raw string interpolation – prevents SQL injection

```

6. Database Schema

Tables Overview



└─ joined_at	
phone_otps	_____
└─ id (UUID PK)	
└─ phone	
└─ otp (6-digit code)	
└─ expires_at	
└─ used (BOOLEAN)	
device_locations	_____
└─ id (UUID PK)	
└─ user_id (FK → users.id)	
└─ latitude (DOUBLE PRECISION)	
└─ longitude (DOUBLE PRECISION)	
└─ accuracy	
└─ battery_level	
└─ speed	
└─ recorded_at	
safe_zones (PostGIS)	_____
└─ id (UUID PK)	
└─ circle_id (FK → circles.id)	
└─ name	
└─ geom (GEOMETRY - polygon circle via ST_Buffer)	
└─ radius_meters (50-50,000)	
└─ created_by (FK → users.id)	
└─ created_at	
geofence_events	_____
└─ id (UUID PK)	
└─ user_id (FK → users.id)	
└─ safe_zone_id (FK → safe_zones.id)	
└─ event_type (entry/exit)	
└─ geom (GEOMETRY - point where event occurred)	
└─ created_at	
sos_events	_____
└─ id (UUID PK)	
└─ user_id (FK → users.id)	
└─ circle_id (FK → circles.id)	
└─ message (TEXT)	
└─ latitude	
└─ longitude	
└─ resolved (BOOLEAN, default false)	
└─ created_at	
user_latest_locations (VIEW)	_____
└─ Materialized view: latest location per user_id	

PostGIS Spatial Operations

Safe zones are stored as geographic buffer circles:

```
-- Creating a safe zone (150-meter radius around a point)
INSERT INTO safe_zones (circle_id, name, geom, radius_meters, created_by)
VALUES (
  $1, $2,
  ST_Buffer(
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography,
    $radius_meters
 )::geometry,
  $radius_meters, $user_id
)

-- Retrieving zone center from geometry
SELECT
  ST_X(ST_Centroid(geom)) as center_lng,
  ST_Y(ST_Centroid(geom)) as center_lat,
  ST_AsGeoJSON(geom)::json as geometry
FROM safe_zones
```

All spatial data uses **SRID 4326** (WGS84 — standard GPS coordinate system).

7. Database Indexes & Performance

Indexes Applied

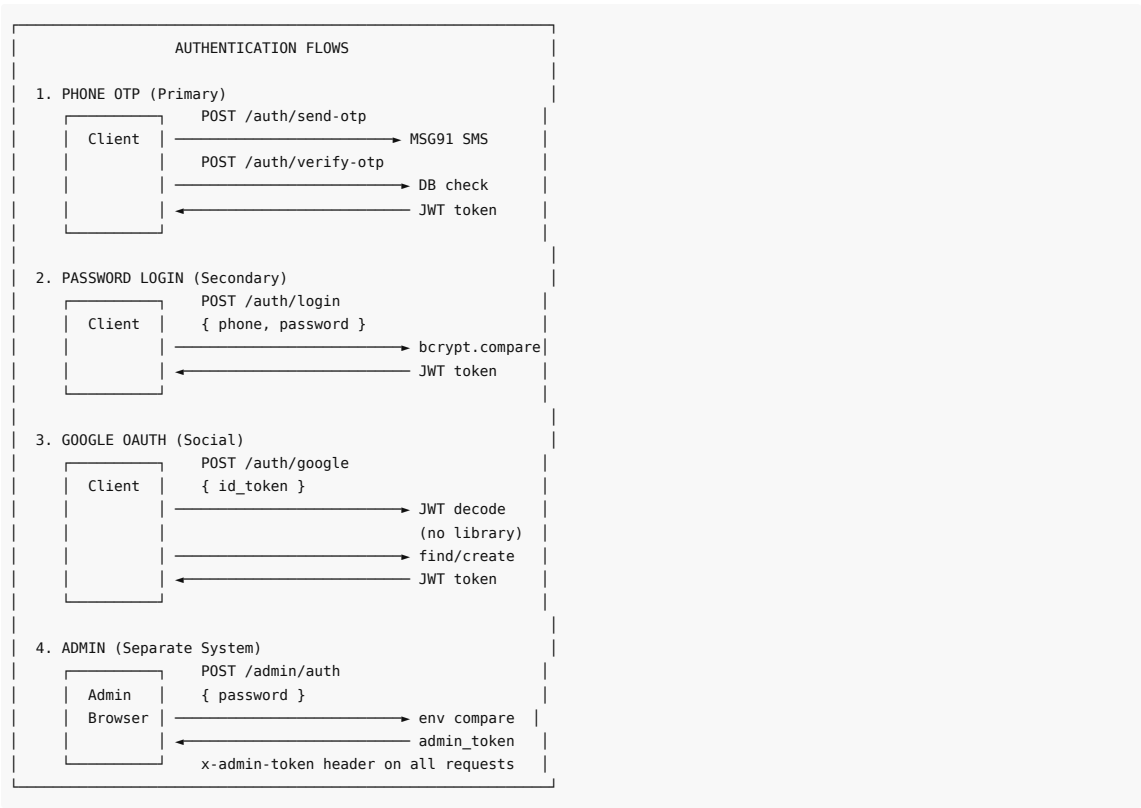
Table	Column(s)	Index Type	Purpose
users	phone	UNIQUE B-tree	OTP lookup, login
users	email	B-tree	Google OAuth lookup
circles	invite_code	UNIQUE B-tree	Join-by-code lookup
circle_members	(circle_id, user_id)	Composite B-tree	Membership check (runs on every authenticated request)
device_locations	(user_id, recorded_at DESC)	B-tree	Location history queries
safe_zones	geom	GIST	PostGIS spatial queries (ST_Within)
geofence_events	(safe_zone_id, created_at DESC)	B-tree	Event log pagination
sos_events	(circle_id, created_at DESC)	B-tree	SOS history pagination
phone_otp	(phone, used, expires_at)	Composite B-tree	OTP verification

Query Performance Notes

- circle_members check** runs on every circle-data API call. The composite index (circle_id, user_id) makes this an O(log n) lookup even with millions of memberships.
- PostGIS GIST index** on safe_zones.geom is critical — without it, geofence entry/exit detection would do a full table scan on every location update.
- device_locations** grows unboundedly over time. The Admin Panel "Purge Old Locations" function (DELETE /admin/purge-locations) deletes records older than a configurable number of days. Recommended: schedule monthly via cron.
- Connection pool** is set to max: 20 connections. At peak load (20 concurrent DB queries), new requests queue until a connection is free. Increase max if DB server has capacity.

8. Authentication System

Authentication Methods



JWT Token Details

```
// Token payload
{
  userId: "uuid-string",
  iat: 1234567890, // issued at
```

```

    exp: 1234567890,    // expiry (configurable via JWT_EXPIRES_IN)
  }

  // Signing
  jwt.sign({ userId }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN // e.g. '7d'
  })

  // Storage
  // Web: localStorage key 'gravity_token'
  // Mobile: AsyncStorage key 'auth_token'
  // Admin: localStorage key 'admin_token' (separate)

```

OTP Rate Limiting

- Maximum **3 OTP requests per phone number per 10 minutes**
- OTPs expire after **10 minutes**
- Each OTP is single-use (marked `used = true` after verification)
- Fallback: if `MSG91_API_KEY` is not set, OTP is printed to server console (development mode)

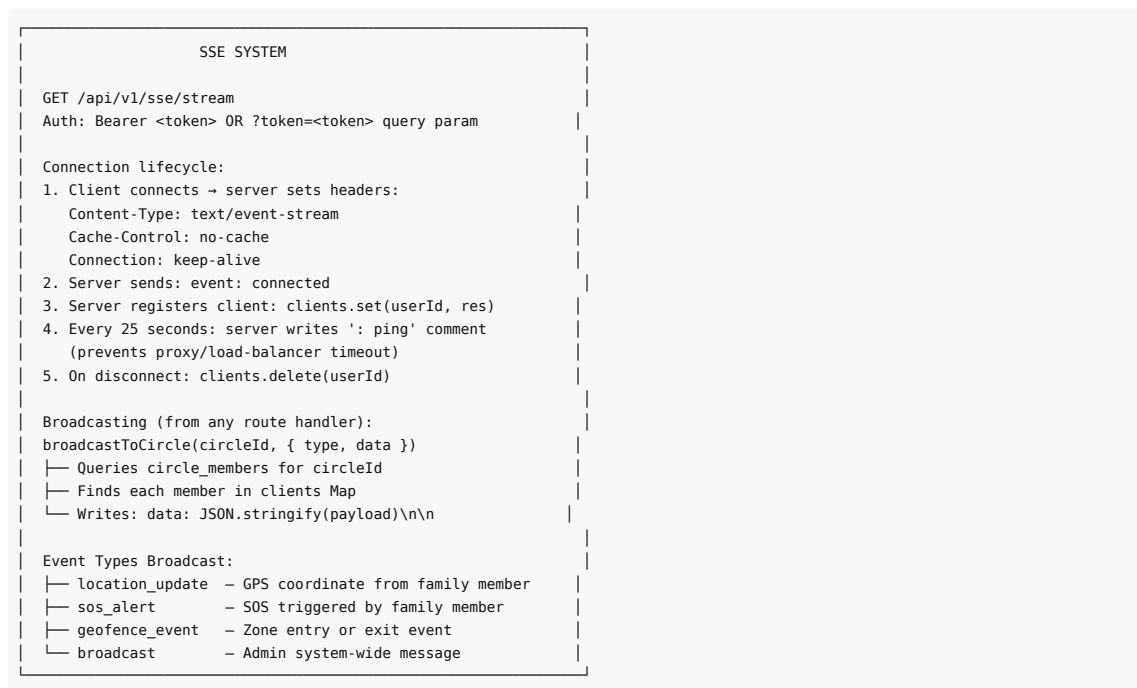
Password Security

- Passwords hashed with `bcrypt` (10 salt rounds)
- `bcrypt.compare()` used for all password verifications
- Passwords never stored in plaintext or logs

9. Real-Time Communication (SSE)

Server-Sent Events Architecture

Gravity uses Server-Sent Events (SSE) instead of WebSockets for real-time updates. SSE is a one-directional push channel from server to client over a persistent HTTP connection.



SSE Client Registry

```

// In-memory Map (per process):
const clients = new Map() // userId → res (Express response object)

// Add client on connect
clients.set(userId, res)

// Remove on disconnect
req.on('close', () => clients.delete(userId))

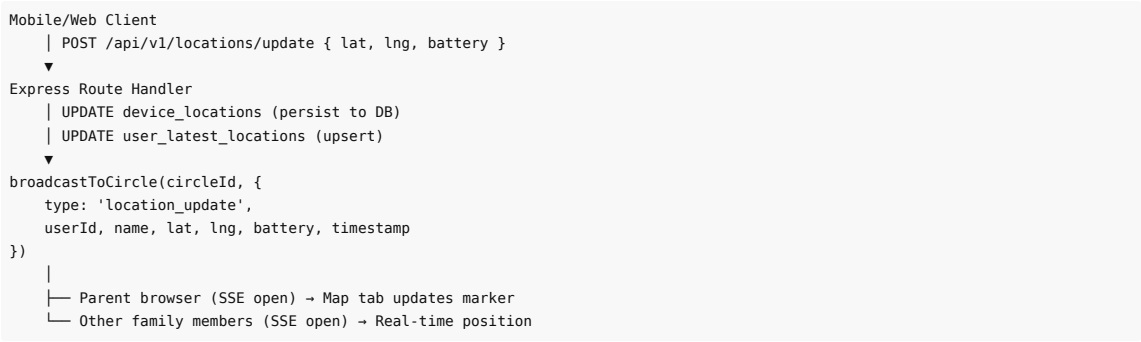
// Broadcast to specific user
function sendToUser(userId, payload) {

```



```
const client = clients.get(userId)
if (client) {
  client.write(`data: ${JSON.stringify(payload)}\n\n`)
}
}
```

Real-Time Location Update Flow



10. File Storage — Cloudflare R2

Avatar Upload Flow (3-Step Presigned Process)



R2 Configuration

- Bucket name from CLOUDFLARE_R2_BUCKET environment variable
- Account ID from CLOUDFLARE_ACCOUNT_ID
- Access credentials: CLOUDFLARE_R2_ACCESS_KEY_ID + CLOUDFLARE_R2_SECRET_ACCESS_KEY
- Public URL prefix: CLOUDFLARE_R2_PUBLIC_URL
- Files are publicly readable once uploaded
- No file size limit enforced at API level (enforced by client)

11. Web Application Architecture

Application Entry



Routing Structure

```

/ (root)
├─ /                → Home.tsx      (Landing page)
├─ /login           → Login.tsx     (Phone OTP login)
├─ /admin/login     → AdminLogin.tsx
├─ /parent/panel    → ParentPanel.tsx (Auth-guarded)
├─ /child/panel     → ChildPanel.tsx (Auth-guarded)
└─ /admin/panel     → AdminPanel.tsx (Admin auth-guarded)

```

Parent Panel — Architecture

```

ParentPanel.tsx
├─ State
│  └─ activeTab: 'map' | 'family' | 'alerts' | 'geofence' | 'settings'
│  └─ members[]      → circle members with live location data
│  └─ sosEvents[]    → real-time SOS alert list
│  └─ geofenceEvents[] → entry/exit event list
│  └─ safeZones[]    → all zones for this circle
│  └─ allCircles[]   → multi-circle support (switcher UI)
│  └─ circleId       → active circle UUID
│  └─ showHistory    → location history overlay
│  └─ historyPoints[] → GPS trail points (last 50)
├─ Real-time (SSE)
│  └─ EventSource('/api/v1/sse/stream?token=...')
│     └─ location_update → update member in state
│     └─ sos_alert       → prepend to sosEvents
│     └─ geofence_event  → prepend to geofenceEvents
├─ Map Tab
│  └─ Leaflet MapContainer
│  └─ 4 tile layer options: Dark (CartoDB) / Light / Satellite / Street
│  └─ Family member markers (custom avatar circles)
│  └─ Safe zone polygons (green circles with labels)
│  └─ Location history polyline (dashed green trail)
│  └─ Multi-circle switcher tabs (shown if >1 circle)
├─ Family Tab → member cards with battery, status, last-seen
├─ Alerts Tab → SOS + geofence events feed with real-time updates
├─ Geofence Tab → safe zone list + create/edit/delete modals
└─ Settings Tab → avatar upload (R2), name edit, logout

```

Child Panel — Architecture

```

ChildPanel.tsx
├─ 6 tabs: home | map | sos | family | alerts | profile
├─
├─ Home Tab → stats overview, SOS shortcut
├─ Map Tab  → Leaflet map, own location + family positions
├─ SOS Tab  → Large SOS button → POST /sos/trigger → SSE broadcast
├─ Family Tab → circle members list
├─ Alerts Tab → All / Geofence / SOS filter chips
└─ Profile Tab → avatar upload, name/email/phone display, edit, logout

```

Admin Panel — Architecture

```

AdminPanel.tsx (1530+ lines)
├─ Dual View Mode System
│  └─ Desktop Mode (default): 240px sidebar + full-width content
│  └─ Mobile Mode: phone-frame (max 393px) + bottom nav
│  └─ Toggled via localStorage key 'admin_view_mode'
├─ 7 Tabs
│  └─ Dashboard → 9-stat grid + recent SOS (shimmer loading)
│  └─ Users     → search + role/status filter + ban/unban/delete
│  └─ Circles   → search + member count + invite regenerate + delete
│  │            + Create Circle FAB (slide-up bottom sheet modal)
│  └─ SOS       → filter (All/New/Resolved) + bulk resolve
│  └─ Logs      → sub-tabs: Geofence events | OTP logs
│  └─ System    → DB stats, server info, SSE count, danger zone
│  └─ Broadcast → type selector + textarea + send + history
├─
└─ Desktop Sidebar
   └─ GRAVITY logo + tagline
   └─ 7 nav items with active highlight
   └─ SOS badge (unresolved count)

```

- └ Mobile View toggle button
- └ Sign Out button

CSS Design System

```
/* Core Color Palette */
--bg-primary:    #050C08 /* Deep black-green background */
--bg-secondary:  #0A1510 /* Slightly lighter surface */
--bg-glass:       rgba(10, 92, 53, 0.08) /* Glass card effect */
--accent:         #00E676 /* Bright green – primary action */
--accent-dark:    #0A5C35 /* Dark green – secondary accent */
--danger:         #FF1744 /* SOS / error / danger red */
--text-primary:   #FFFFFF /* Primary text */
--text-muted:     #7A9E8A /* Secondary text */
--border:         rgba(0, 230, 118, 0.12) /* Subtle green border */

/* CSS Module keyframes */
@keyframes fadeIn { 0% → opacity 0 | 100% → opacity 1 }
@keyframes shimmer { shimmer loading skeleton effect }
@keyframes sosPulse { red pulsing ring for SOS alerts }
@keyframes slideUp { panel slide-in from bottom }
@keyframes ping { expanding circle pulse }
@keyframes toastIn { notification slide-in }
```

12. Mobile Application Architecture

App Structure

```
mobile/
├ App.js                ← Root component, NavigationContainer
├ src/
│ ├── navigation/
│ │ ├── TabNavigator.jsx ← Bottom tabs (blur background, iOS/Android)
│ │ └ RootNavigator.jsx  ← Auth guard (Login vs Tabs)
│ ├── screens/
│ │ ├── AuthScreen.jsx   ← Phone + OTP login
│ │ ├── MapScreen.jsx    ← Live family map (React Native Maps)
│ │ ├── CirclesScreen.jsx ← Circle management
│ │ ├── SafeZonesScreen.jsx ← Geofence zone viewer
│ │ ├── AlertsScreen.jsx ← SOS + geofence alerts
│ │ └ ProfileScreen.jsx  ← User profile + settings
│ ├── services/
│ │ ├── api.js           ← Full API client (all endpoints)
│ │ ├── location.js      ← Background GPS tracking
│ │ ├── notifications.js ← Expo push notification setup
│ │ └ offlineQueue.js    ← Queue for poor connectivity
│ ├── components/
│ │ └ (reusable UI components)
│ └ theme/
│   └ colors.js          ← Color constants (matches web theme)
```

Bottom Tab Navigation

```
Tab Bar (5 Tabs)
├ Map      → MapScreen.jsx
├ Circles  → CirclesScreen.jsx
├ Zones    → SafeZonesScreen.jsx
├ Alerts   → AlertsScreen.jsx
└ Profile  → ProfileScreen.jsx
```

Tab Bar Style:

- Position: absolute (floats over content)
- Background: iOS → `BlurView` (blur effect) | Android → semi-opaque dark
- Border top: green (`Colors.border`)
- Active icon: Ionicons solid + green tint
- Inactive icon: Ionicons outline + muted gray
- Active icon background: glass highlight square

MapScreen Architecture

```
MapScreen.jsx (781 lines)
├ State
│ ├── ownLocation    ← device GPS (expo-location)
│ └ familyMembers[] ← polled every 10 seconds from API
```

- selectedMember - tapped marker → info card
- sosVisible - SOS confirm modal
- Location Tracking
 - expo-location.watchPositionAsync (HIGH_ACCURACY)
 - Updates own marker every position change
 - POST /api/v1/locations/update every 30 seconds
- Map Display
 - react-native-maps MapView
 - PROVIDER_GOOGLE on Android, default on iOS
 - DARK_MAP_STYLE (custom JSON style array)
 - Own location: green animated pulsing dot (Animated.loop)
 - Family markers: circular avatar with initials + name label
- Interactions
 - Tap family marker → slide-up info card
 - Shows: name, role, battery %, last-seen timestamp
 - SOS FAB (bottom-right) → confirm dialog → POST /sos/trigger
- SOS Trigger
 - POST /api/v1/sos/trigger { circleId, message, lat, lng }
 - Server SSE broadcast to all members
 - Server sends Expo push notifications

Service Layer

api.js — API Client

```
// Base URL from environment
const BASE_URL = process.env.EXPO_PUBLIC_API_URL || 'https://gravity.trackalways.com'

// Auth token stored in AsyncStorage
const getToken = () => AsyncStorage.getItem('auth_token')

// All API methods return { data } or throw error
// Methods cover: auth, users, circles, sos, geofences, locations, media
```

location.js — Background Location

```
// Permissions: foreground (always) + background (when-in-use)
// Update interval: 30 seconds
// Accuracy: LocationAccuracy.High
// Distance filter: 10 meters (only sends if moved >10m)
// Sends: { latitude, longitude, accuracy, battery_level }
```

notifications.js — Push Setup

```
// On app launch:
// 1. Request notification permissions
// 2. Get Expo Push Token
// 3. POST /api/v1/users/me/push-token { push_token }
// 4. Listen for incoming notifications (foreground)
// 5. Handle notification tap (background/killed)
```

offlineQueue.js — Connectivity Resilience

```
// Queue location updates when offline
// Retry on reconnect (NetInfo listener)
// Max queue size: 100 entries
// Each entry: { endpoint, payload, timestamp }
```

13. Mobile App Permissions

The Expo app declares the following device permissions in `app.json` . These are shown to users during installation or at first use.

Android Permissions (`AndroidManifest.xml` via Expo)

Permission	When Requested	Purpose
ACCESS_FINE_LOCATION	On Map screen first open	GPS tracking (high accuracy)
ACCESS_COARSE_LOCATION	Fallback if fine denied	Approximate location

ACCESS_BACKGROUND_LOCATION	After fine location granted	Sends location when app is in background
RECEIVE_BOOT_COMPLETED	Install time	Restart location service after device reboot
VIBRATE	Runtime	Vibrate on SOS alert received
POST_NOTIFICATIONS	Android 13+ runtime	Show push notifications

iOS Permissions (Info.plist via Expo)

Permission Key	Shown Text	Purpose
NSLocationWhenInUseUsageDescription	"Gravity needs your location to share with family members"	Map screen
NSLocationAlwaysAndWhenInUseUsageDescription	"Allow background location to keep family updated even when app is closed"	Background tracking
NSLocationAlwaysUsageDescription	Same as above	iOS 10 compatibility

Permission Request Flow

```

App opens for first time
├── Map Screen → requestForegroundPermissionsAsync()
│   ├── Granted → start location watching
│   └── Denied → show "Location required" message, limit features
├── On login → requestPermissionsAsync() (notifications)
│   ├── Granted → register Expo push token with backend
│   └── Denied → SOS push alerts will not work (warn user)
└── Background location (Android only)
    ├── requestBackgroundPermissionsAsync()
    │   ├── Granted → location updates when app backgrounded
    │   └── Denied → location only updates when app is open

```

14. Push Notifications

Expo Push Notification Flow

```

Family Member (Mobile)
├── SOS triggered on device
├── POST /api/v1/sos/trigger { circleId, message, lat, lng }
└──
    ▼
Express SOS Route Handler
├── 1. INSERT INTO sos_events
├── 2. broadcastToCircle() → SSE to all web/mobile clients
├── 3. SELECT push_token FROM users WHERE id IN (circle members)
├── 4. Filter: tokens must start with 'ExponentPushToken['
└──
    ▼
Expo Push API (https://exp.host/--/api/v2/push/send)
├── Chunked requests (max 100 tokens per request)
├── Payload: { to, title, body, data: { type: 'sos', circleId } }
└──
    ▼
Apple APNs / Google FCM
├──
└──
    ▼
Family Members' Devices
└── Push notification appears even if app is in background/closed

```

Push Token Registration

On every app login or startup, the mobile app:

1. Calls `Notifications.getExpoPushTokenAsync()`
2. POSTs the token to `PATCH /api/v1/users/me/push-token`
3. Token stored in `users.push_token` column

15. Geofencing System

Zone Creation

```

-- Zone stored as a geographic buffer circle
INSERT INTO safe_zones (circle_id, name, geom, radius_meters, created_by)
VALUES ($circle_id, $name,
    ST_Buffer(

```

```
ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography,
$radius_meters
)::geometry,
$radius_meters, $user_id
)
```

Entry/Exit Detection

When a device posts a location update:

1. Backend checks if the new position intersects any safe zones in the user's circle
2. Compares with previous position's zone membership
3. If zone membership changed → INSERT into `geofence_events`
4. Broadcast geofence event via SSE to all circle members

```
-- Check if point is inside any safe zone
SELECT sz.id, sz.name
FROM safe_zones sz
WHERE sz.circle_id = $circleId
AND ST_Within(
  ST_SetSRID(ST_MakePoint($lng, $lat), 4326),
  sz.geom
)
```

Geofence API

Endpoint	Method	Description
/api/v1/geofences/circle/:circleId	GET	List all safe zones
/api/v1/geofences/events/:circleId	GET	Entry/exit event log
/api/v1/geofences/	POST	Create new safe zone
/api/v1/geofences/:id	PATCH	Edit zone name/position/radius
/api/v1/geofences/:id	DELETE	Delete zone

16. Security Model

Defense Layers

Layer 1: Transport Security

- └ TLS 1.3 via Caddy (automatic Let's Encrypt)
- └ HSTS enforced by Caddy

Layer 2: Network Controls

- └ Security headers (X-Frame-Options: DENY, etc.)
- └ Rate limiting: 1000 requests / 15 minutes per IP
- └ OTP rate limit: 3 OTPs / 10 minutes per phone
- └ CORS: Configured per environment

Layer 3: Authentication

- └ JWT with configurable expiry (default 7 days)
- └ JWT_SECRET from environment (never hardcoded)
- └ Separate admin token (x-admin-token header)
- └ bcrypt password hashing (10 rounds)
- └ Google OAuth: id_token decoded without external library

Layer 4: Authorization

- └ All user routes require authenticate() middleware
- └ Circle data access: circle_members table checked on every query
- └ Admin routes: separate x-admin-token header check
- └ Geofence/zone mutations: role checked (admin vs member)

Layer 5: Data Validation

- └ Zod schemas on all POST/PATCH request bodies
- └ UUID format validated for all ID parameters
- └ Parameterized SQL queries (no string interpolation)
- └ Number range checks (radius: 50–50,000 meters)

Layer 6: Storage

- └ Passwords: bcrypt hashed, never logged
- └ OTPs: single-use, time-limited, stored hashed
- └ Avatar files: uploaded directly to R2 (never through API server)
- └ Push tokens: stored per-user, not exposed in responses

Admin Security

The admin panel uses a completely separate authentication system:

- No JWT; uses a static `ADMIN_TOKEN` from environment
- Sent as `x-admin-token` header on every admin API request
- Separate localStorage key (`admin_token`) from user tokens
- Admin panel accessible at `/admin/panel` (separate route)

17. API Endpoint Reference

Base URL: `https://gravity.trackalways.com/api/v1`

Auth: All user endpoints require `Authorization: Bearer <jwt_token>` header

Admin Auth: Admin endpoints require `x-admin-token: <admin_token>` header

Authentication — /auth

Method	Path	Auth	Description
POST	<code>/auth/send-otp</code>	None	Send OTP to phone (MSG91)
POST	<code>/auth/verify-otp</code>	None	Verify OTP → return JWT
POST	<code>/auth/register</code>	None	Register new user with OTP
POST	<code>/auth/login</code>	None	Login with phone + password
POST	<code>/auth/google</code>	None	Login with Google id_token
POST	<code>/auth/refresh</code>	Bearer	Refresh JWT token

Users — /users

Method	Path	Auth	Description
GET	<code>/users/me</code>	Bearer	Get own profile
PATCH	<code>/users/me</code>	Bearer	Update name / avatar_url
POST	<code>/users/me/location</code>	Bearer	Post current GPS location
POST	<code>/users/me/battery</code>	Bearer	Post battery level
POST	<code>/users/me/push-token</code>	Bearer	Register Expo push token
GET	<code>/users/me/stats</code>	Bearer	Get personal stats
GET	<code>/users/me/location-history</code>	Bearer	Get last N location points

Circles — /circles

Method	Path	Auth	Description
POST	<code>/circles</code>	Bearer	Create new circle
POST	<code>/circles/join</code>	Bearer	Join circle with invite code
GET	<code>/circles/my</code>	Bearer	Get all circles user belongs to
GET	<code>/circles/:id/members</code>	Bearer	Get circle member list
DELETE	<code>/circles/:id/members/:userId</code>	Bearer	Remove member from circle

SOS — /sos

Method	Path	Auth	Description
POST	<code>/sos/trigger</code>	Bearer	Trigger SOS alert + SSE broadcast + push
GET	<code>/sos/history</code>	Bearer	Get SOS event history for circle
PATCH	<code>/sos/:id/resolve</code>	Bearer	Mark SOS as resolved

Geofences — /geofences

Method	Path	Auth	Description
GET	<code>/geofences/circle/:circleId</code>	Bearer	List safe zones
GET	<code>/geofences/events/:circleId</code>	Bearer	Entry/exit event log
POST	<code>/geofences</code>	Bearer	Create new safe zone

PATCH	/geofences/:id	Bearer	Edit safe zone
DELETE	/geofences/:id	Bearer	Delete safe zone

Locations — /locations

Method	Path	Auth	Description
POST	/locations/update	Bearer	Post location + trigger SSE broadcast
GET	/locations/circle/:circleId	Bearer	Get latest location for all members

Media — /media

Method	Path	Auth	Description
GET	/media/avatar-upload-url	Bearer	Get R2 presigned PUT URL

SSE — /sse

Method	Path	Auth	Description
GET	/sse/stream	Bearer token or ?token=	Open SSE stream

Admin — /admin

Method	Path	Admin Auth	Description
POST	/admin/auth	None	Admin login → returns admin_token
GET	/admin/dashboard	x-admin-token	9 platform statistics
GET	/admin/users	x-admin-token	All users with filters
PATCH	/admin/users/:id/ban	x-admin-token	Ban/unban user
DELETE	/admin/users/:id	x-admin-token	Delete user
GET	/admin/circles	x-admin-token	All circles with stats
POST	/admin/circles	x-admin-token	Create circle by owner phone
PATCH	/admin/circles/:id/invite-code	x-admin-token	Regenerate invite code
DELETE	/admin/circles/:id	x-admin-token	Delete circle
GET	/admin/sos	x-admin-token	All SOS events with filters
PATCH	/admin/sos/:id/resolve	x-admin-token	Resolve SOS event
GET	/admin/geofence-events	x-admin-token	All geofence events
GET	/admin/otps	x-admin-token	OTP log with phone search
GET	/admin/system	x-admin-token	DB size, uptime, SSE count
POST	/admin/broadcast	x-admin-token	Broadcast SSE message to all
DELETE	/admin/purge-locations	x-admin-token	Delete old location data

18. API Error Response Format

All error responses from the API follow a consistent JSON structure:

```
{
  "error": "Human-readable error message"
}
```

HTTP Status Codes Used

Code	Meaning	When Returned
200 OK	Success	GET / PATCH / DELETE success
201 Created	Resource created	POST success (new user, circle, zone, etc.)
400 Bad Request	Validation failed	Zod schema mismatch, invalid UUID, missing required field
401 Unauthorized	Auth required	Missing token, expired token, wrong admin password
403 Forbidden	Access denied	User not a member of this circle, insufficient role

404 Not Found	Resource missing	User/circle/zone ID does not exist
429 Too Many Requests	Rate limit hit	>1000 requests/15 min (IP) or >3 OTPs/10 min (phone)
500 Internal Server Error	Server error	Unhandled DB errors, unexpected exceptions

Validation Error Example (400)

When Zod validation fails, the error message lists which fields failed:

```
{
  "error": "Validation failed",
  "details": [
    { "field": "radius_meters", "message": "Number must be at least 50" },
    { "field": "circle_id", "message": "Invalid uuid" }
  ]
}
```

Success Response Examples

```
// GET /api/v1/users/me
{
  "user": {
    "id": "a3f9c1d2-...",
    "name": "Rahul Sharma",
    "phone": "+919876543210",
    "email": "rahul@gmail.com",
    "role": "parent",
    "avatar_url": "https://pub-xxx.r2.dev/avatars/a3f9c1d2.jpg",
    "is_banned": false,
    "created_at": "2026-01-15T10:30:00Z"
  }
}

// POST /api/v1/sos/trigger
{
  "sos": {
    "id": "b7e2a8f1-...",
    "circle_id": "c4d9e3f2-...",
    "message": "Help needed!",
    "latitude": 28.6139,
    "longitude": 77.2090,
    "resolved": false,
    "created_at": "2026-06-19T14:22:11Z"
  }
}
```

19. API Request & Response Examples

Send OTP

POST /api/v1/auth/send-otp
Content-Type: application/json

```
{ "phone": "+919876543210" }
```

```
{ "message": "OTP sent successfully" }
```

Verify OTP + Get JWT

POST /api/v1/auth/verify-otp
Content-Type: application/json

```
{ "phone": "+919876543210", "otp": "847291" }
```

```
{
  "verified": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..",
  "user": { "id": "...", "name": "Rahul", "role": "parent" }
}
```

Post Location Update

```
POST /api/v1/locations/update
Authorization: Bearer <token>
Content-Type: application/json

{
  "latitude": 28.6139,
  "longitude": 77.2090,
  "accuracy": 12.5,
  "battery_level": 78,
  "circle_id": "c4d9e3f2-..."
}
```

```
{ "success": true, "broadcast": true }
```

Create Safe Zone

```
POST /api/v1/geofences
Authorization: Bearer <token>
Content-Type: application/json

{
  "circle_id": "c4d9e3f2-...",
  "name": "Home",
  "center_lat": 28.6139,
  "center_lng": 77.2090,
  "radius_meters": 200
}
```

```
{
  "safe_zone": {
    "id": "d8f3b2e1-...",
    "name": "Home",
    "radius_meters": 200,
    "center_lat": 28.6139,
    "center_lng": 77.2090,
    "created_at": "2026-06-19T10:00:00Z"
  }
}
```

SSE Stream Event Payload (received by client)

```
data:
{"type":"location_update","userId":"a3f9...", "name":"Rahul", "latitude":28.6139, "longitude":77.2090, "battery":78, "timestamp":"2026-06-19T14:22:11Z"}
```

```
data:
{"type":"sos_alert","userId":"b7e2...", "name":"Priya", "latitude":28.55, "longitude":77.21, "message":"Help!", "sosId":"x9y1..."}
```

```
data: {"type":"geofence_event","userId":"a3f9...", "name":"Rahul", "eventType":"exit", "zoneName":"Home", "zoneId":"d8f3..."}
```

20. Data Flow Diagrams

SOS Alert Complete Flow



```

|   |   | Parent browser (SSE) → Alert tab flashes red
|   |   | Admin panel (SSE) → Dashboard SOS count ++
|
|   | 3. SELECT push_token FROM users
|   |   WHERE id IN (SELECT user_id FROM circle_members
|   |                   WHERE circle_id = $circleId)
|   |   AND push_token IS NOT NULL
|
|   | 4. POST https://exp.host/--/api/v2/push/send
|   |   { to: [push_tokens], title: '🚨 SOS Alert',
|   |     body: 'userName needs help!',
|   |     data: { type: 'sos', circleId, lat, lng } }
|   |   | Apple APNs + Google FCM delivery

```

Live Location Update Flow

```

User Device (Mobile)
|
|   expo-location watchPositionAsync fires
|   every 30s or when moved >10m
|
|   POST /api/v1/locations/update
|   { latitude, longitude, accuracy, battery_level }
|
▼
Express Locations Route
|
|   | INSERT device_locations (full history)
|   | UPSERT user_latest_locations (for fast queries)
|   |
|   | broadcastToCircle(circleId, {
|   |   type: 'location_update',
|   |   userId, lat, lng, battery, timestamp
|   | })
|   |
|   | Parent web (SSE open) → Leaflet marker moves
|   | Other mobile (SSE open) → Map marker moves
|   | Admin panel → SSE client count visible in System tab

```

User Registration Flow

```

New User (Mobile/Web)
|
|   Step 1: POST /auth/send-otp { phone }
|
▼
Express Auth Route
|
|   | Check OTP rate limit (max 3 in 10 min)
|   | Generate 6-digit OTP
|   | INSERT phone_otps { phone, otp, expires_at }
|   | MSG91 API call → SMS to phone
|   |   | User receives SMS
|   |   |
|   |   ▼
|   |   Step 2: POST /auth/verify-otp { phone, otp }
|   |   |
|   |   | SELECT FROM phone_otps WHERE phone=$1 AND used=false
|   |   | Check expiry + code match
|   |   | UPDATE phone_otps SET used=true
|   |   | Return { verified: true }
|   |   |
|   |   | Registration screen shown
|   |   |
|   |   ▼
|   |   Step 3: POST /auth/register { phone, name, role, password }
|   |   |
|   |   | INSERT users { phone, name, role, password_hash }
|   |   | jwt.sign({ userId }) → return JWT token
|   |   |
|   |   |
|   |   | Client stores token
|   |   |   | localStorage 'gravity_token' (web)
|   |   |   | AsyncStorage 'auth_token' (mobile)

```

21. Third-Party Service Dependencies

Service	Used For	Fallback if Unavailable
---------	----------	-------------------------

MSG91	SMS OTP delivery	If MSG91_API_KEY not set, OTP is printed to server console (dev mode). In production: user cannot receive OTP → login fails.
Cloudflare R2	Avatar image storage	If R2 is down, avatar upload fails but all other features continue. Avatars fall back to initials avatar in UI.
Expo Push API	Mobile push notifications	If Expo API is unreachable, SOS push is silently skipped. SSE alert still reaches web users. No retry implemented.
Apple APNs	iOS push delivery	Managed by Expo — no direct dependency. Expo handles APNs/FCM routing.
Google FCM	Android push delivery	Same as above — managed by Expo.
Traccar	GPS device telemetry ingestion	Optional service. Core tracking via direct API still works if Traccar is offline.
Let's Encrypt (via Caddy)	TLS certificates	Caddy auto-renews 30 days before expiry. If renewal fails, HTTPS breaks. Monitor cert expiry.
Google Maps API	Mobile map tiles (Android)	App requires GOOGLE_MAPS_API_KEY in app.json. Without it, Android MapView shows blank. iOS uses Apple Maps (no key needed).
CartoDB tile server	Web map tiles (Leaflet dark mode)	If CartoDB is down, dark tile style shows blank. User can switch to Street/Satellite tile in UI.

22. Scalability & Known Limitations

Current Architecture Limits

Limitation	Detail	How to Scale
SSE in-memory Map	The clients Map lives in the single Node.js process. If the API is horizontally scaled (multiple instances), a user on server A won't receive broadcasts from server B.	Add Redis Pub/Sub as a shared broadcast channel between instances.
Single PM2 instance	instances: 1 in ecosystem.config.js — no clustering.	Change to instances: 'max' for multi-core utilization. Requires Redis for SSE (see above).
device_locations table growth	Every location update is stored permanently. At 1 location/30s per user, 1000 active users = ~2.9M rows/day.	Run monthly purge (DELETE /admin/purge-locations), or add pg_partman time-based partitioning.
Connection pool cap	max: 20 DB connections. Under 20 concurrent queries, performance is fine. Beyond that, queries queue.	Increase max if server has a large PostgreSQL max_connections. Add PgBouncer for 1000+ concurrent users.
No WebSocket	SSE is one-directional (server → client only). Clients send location via REST POST, not via SSE. This adds latency vs full-duplex WebSocket but is simpler and HTTP-compatible.	Acceptable at current scale. Migrate to WebSocket if sub-second bidirectional updates are needed.
OTP via MSG91 only	No secondary SMS provider. If MSG91 has an outage, OTP delivery fails entirely.	Add Twilio or Fast2SMS as fallback provider.
No message queue	Push notifications are sent synchronously within the SOS route handler. If Expo API is slow, it adds latency to the SOS response.	Move push notifications to a background job queue (Bull + Redis) for decoupled delivery.

Expected Capacity (Single Server)

Metric	Estimate
Concurrent SSE connections	~500 (limited by server RAM and file descriptors)
API requests/second	~200 (1000 req/15min rate limit per IP; server can handle far more without rate limit)
Active users per circle	No limit enforced; practical limit ~50 before SSE broadcast becomes slow
Safe zones per circle	No limit enforced; PostGIS handles thousands efficiently

23. Monitoring & Health Checks

Health Check Endpoint

The Admin Panel's System tab (GET /api/v1/admin/system) serves as the health check, returning:

```
{
  "database": {
    "size_mb": 45.2,
    "tables": { "users": 312, "device_locations": 891230 }
  },
}
```

```
"server": {
  "uptime_seconds": 1209600,
  "node_version": "v20.20.0",
  "memory_usage_mb": 128
},
"sse": {
  "active_connections": 23
}
}
```

Process Monitoring — PM2

```
pm2 status          # View all process statuses
pm2 monit            # Real-time CPU + memory dashboard
pm2 logs gravity-api # Tail API logs
pm2 logs --lines 200 # Last 200 lines all processes
```

PM2 auto-restarts any process that:

- Exits unexpectedly (crash)
- Exceeds memory limit (500MB for API, 200MB for web server)

Log Files

Log	Path	Format
API stdout	/tmp/gravity-api-out.log	Plain text with timestamps
API errors	/tmp/gravity-api-error.log	Node.js stack traces
Caddy access	/var/log/caddy/gravity.log	JSON (includes IP, path, status, duration)

Recommended External Monitoring (Not Yet Implemented)

Tool	Purpose
UptimeRobot (free)	HTTP uptime check every 5 minutes — alerts via email/SMS if API is down
PM2 Plus	Cloud dashboard for PM2 metrics, restart alerts
Sentry	Error tracking — captures unhandled exceptions in Express with user context
Grafana + Prometheus	Metrics dashboards for DB query times, SSE connection count, request rates

24. Backup & Disaster Recovery

Database Backup

Current Setup: No automated backup configured. Recommended setup:

```
# Daily PostgreSQL dump (add to cron)
pg_dump -U gravity_user -F c gravity_db > /backups/gravity_$(date +%Y%m%d).dump

# Restore from backup
pg_restore -U gravity_user -d gravity_db /backups/gravity_20260619.dump
```

Recommended backup schedule:

- Full dump: Daily at 2 AM
- Retention: 7 daily + 4 weekly + 3 monthly
- Offsite storage: Copy to Cloudflare R2 or AWS S3

What Can Be Lost (Recovery Point Objective)

Data Type	Loss Risk Without Backup	Recovery
User accounts, circles	High impact	Restore from DB dump
Location history	Medium impact (non-critical history)	Restore from DB dump
Avatar images	Low impact	R2 is managed — no backup needed
Safe zones	High impact	Restore from DB dump
Code / Config	No data loss	Git repository

Disaster Recovery Steps

```
1. Provision new server
2. Install Node.js 20, PostgreSQL 15, PostGIS 3, PM2, Caddy
3. Clone repo: git clone https://github.com/kamaralam1984/gravitypro
4. Restore DB from latest dump: pg_restore ...
5. Copy backend/.env from secure storage
6. Run: cd landing-react && npm run build
7. Start: pm2 start ecosystem.config.js
8. Start Caddy: caddy start --config caddy/Caddyfile
9. Verify: curl https://gravity.trackalways.com/api/v1/admin/system
```

Estimated recovery time: **30–60 minutes** with prepared backups.

25. Data Privacy & Compliance

Data Collected

Data Type	Stored In	Sensitivity
Phone number	users.phone	High — personal identifier
Name	users.name	Medium
Email	users.email	High — personal identifier
GPS coordinates (history)	device_locations	Very High — movement patterns
GPS coordinates (latest)	user_latest_locations	Very High
Profile photo	Cloudflare R2	Medium
SOS messages	sos_events.message	High — may contain distress context
Expo push token	users.push_token	Medium — device identifier

Data Isolation

- All location, SOS, and geofence data is scoped to **circles**
- A user can only query data for circles they are a member of (enforced by `circle_members` check on every query)
- Admins can see all data across all circles (admin panel)

Children's Data

- The platform supports "child" role users who may be minors
- Location data of children is visible to their circle (parents)
- No third-party analytics SDK is included in the mobile app
- No advertising or tracking pixels

User Data Deletion

Admin can delete a user account via `DELETE /api/v1/admin/users/:id`. This:

- Removes the `users` row
- Cascades to `circle_members` (user leaves all circles)
- Does **not** automatically delete `device_locations` (orphaned rows)
- Does **not** delete avatar from R2 (manual cleanup required)

Recommendation: Implement a proper user-data-deletion job that cleans up all associated records and R2 files when a user account is deleted.

GDPR / Data Protection Notes

If operating in the EU or UK:

- Provide users with a "Download My Data" feature (not yet implemented)
- Provide users with "Delete My Account" self-service (not yet implemented)
- Add a Privacy Policy page on the landing page
- Document data retention periods (currently indefinite for locations)
- Obtain explicit consent for location tracking on registration

26. Target Markets

The Gravity platform is designed for global use. The Caddy configuration explicitly notes support for users in:

Region	Countries	Notes
South Asia	India	Primary development market; MSG91 OTP service works well
East Africa	Kenya	Supported; MSG91 covers Kenya
Middle East	UAE	Supported
Europe	UK	Supported; GDPR compliance may be required

North America	USA	Supported
---------------	-----	-----------

Language: Platform UI is in English. No i18n/localization is implemented currently.

Currency / Payments: No payment system implemented. Platform operates without monetization. Future: Stripe (global) or Razorpay (India).

27. Development Setup Guide

Prerequisites

- Node.js 20.x
- PostgreSQL 15 with PostGIS extension
- Git

Backend Setup

```
git clone https://github.com/kamaralam1984/gravitypro
cd Gravity/backend
npm install
cp .env.example .env # Fill in all required variables
# Create database
psql -U postgres -c "CREATE DATABASE gravity_db;"
psql -U postgres -d gravity_db -c "CREATE EXTENSION postgis;"
# Run migrations (SQL schema files)
psql -U gravity_user -d gravity_db -f schema.sql
# Start development server
npm run dev # Uses nodemon for auto-reload
```

Frontend Setup

```
cd Gravity/landing-react
npm install
npm run dev # Starts Vite dev server on port 5173
# API calls proxy to localhost:8002 (configured in vite.config.ts)
```

Mobile App Setup

```
cd Gravity/mobile
npm install
# Install Expo CLI
npm install -g @expo/cli
# Set environment
echo "EXPO_PUBLIC_API_URL=http://localhost:8002" > .env
# Start Expo
npx expo start
# Scan QR code with Expo Go app on device
# OR press 'a' for Android emulator, 'i' for iOS simulator
```

Running All Services Locally

```
# Terminal 1 – Backend API
cd backend && npm run dev

# Terminal 2 – Web frontend
cd landing-react && npm run dev

# Terminal 3 – Mobile
cd mobile && npx expo start
```

Building for Production

```
# Build web frontend
cd landing-react && npm run build
# Output in landing-react/dist/

# Start with PM2
pm2 start ecosystem.config.js

# Build mobile (requires Expo EAS CLI)
cd mobile
```

```
npx eas build --platform android
npx eas build --platform ios
```

28. Environment Configuration

Backend — backend/.env

```
# Database
DB_HOST=localhost
DB_PORT=5432
DB_NAME=gravity_db
DB_USER=gravity_user
DB_PASSWORD=<secure-password>

# JWT
JWT_SECRET=<long-random-secret>
JWT_EXPIRES_IN=7d

# Admin
ADMIN_TOKEN=<secure-admin-password>

# SMS OTP
MSG91_API_KEY=<msg91-api-key>
MSG91_TEMPLATE_ID=<template-id>
MSG91_SENDER_ID=GRAVITY

# Cloudflare R2
CLOUDFLARE_ACCOUNT_ID=<account-id>
CLOUDFLARE_R2_ACCESS_KEY_ID=<access-key>
CLOUDFLARE_R2_SECRET_ACCESS_KEY=<secret-key>
CLOUDFLARE_R2_BUCKET=gravity-avatars
CLOUDFLARE_R2_PUBLIC_URL=https://pub-<hash>.r2.dev

# Server
PORT=8002
NODE_ENV=production
```

Mobile — mobile/.env

```
EXPO_PUBLIC_API_URL=https://gravity.trackalways.com
```

Web Frontend — Build-time (Vite)

```
# In landing-react/.env (Vite prefix required)
VITE_API_URL=https://gravity.trackalways.com
```

29. Project File Structure

Gravity/

|

├─ backend/

|

| └─ src/

| |

| | └─ app.js

| | |

| | | └─ config/

| | | |

| | | | └─ db.js

| | | |

| | | └─ middleware/

| | | |

| | | | └─ auth.js

| | | | |

| | | | └─ validate.js

| | | | |

| | | | └─ routes/

| | | | |

| | | | | └─ auth.js

| | | | |

| | | | | └─ users.js

| | | | |

| | | | | └─ circles.js

| | | | |

| | | | | └─ sos.js

| | | | |

| | | | | └─ geofences.js

| | | | |

| | | | | └─ locations.js

| | | | |

| | | | | └─ media.js

| | | | |

| | | | | └─ sse.js

| | | | | |

| | | | | └─ admin.js

| | | └─ package.json

|

└─ landing-react/

├─ Express.js API (Port 8002)

├─ Main entry, middleware setup

├─ PostgreSQL pool connection

├─ JWT authentication

├─ Zod schema validation

├─ OTP, Google OAuth, login

├─ Profile, location, push token

├─ Family group management

├─ SOS alerts + push notifications

├─ Safe zones (PostGIS)

├─ GPS tracking + SSE broadcast

├─ Cloudflare R2 upload URLs

├─ SSE stream handler

├─ Full admin management API

├─ React+Vite Web App (Port 8090)


```

├─ index.html
├─ server.cjs
├─ vite.config.ts
├─ src/
│   ├── main.tsx
│   ├── index.css
│   └─ pages/
│       ├── Home.tsx / .module.css
│       ├── Login.tsx
│       ├── AdminLogin.tsx
│       ├── ParentPanel.tsx / .module.css
│       ├── ChildPanel.tsx / .module.css
│       └─ AdminPanel.tsx / .module.css
└─ dist/

mobile/
├─ App.js
├─ app.json
└─ src/
    ├── navigation/
    │   ├── RootNavigator.jsx
    │   └─ TabNavigator.jsx
    ├── screens/
    │   ├── AuthScreen.jsx
    │   ├── MapScreen.jsx
    │   ├── CirclesScreen.jsx
    │   ├── SafeZonesScreen.jsx
    │   ├── AlertsScreen.jsx
    │   └─ ProfileScreen.jsx
    ├── services/
    │   ├── api.js
    │   ├── location.js
    │   ├── notifications.js
    │   └─ offlineQueue.js
    ├── components/
    ├── theme/
    │   └─ colors.js
    └─ ...

caddy/
├─ Caddyfile

ecosystem.config.js

GRAVITY_SYSTEM_INFO.md
GRAVITY_SYSTEM_INFO.pdf
GRAVITY_ARCHITECTURE.md
PROJECT_REPORT.md

```

Summary

Area	Technology	Status
Backend API	Node.js + Express 5.2.x	Production-ready
Database	PostgreSQL 15 + PostGIS 3	Production-ready
Real-Time	Server-Sent Events (SSE)	Production-ready
Web App	React 18 + TypeScript + Vite	Production-ready
Mobile App	React Native + Expo 54	~80% complete
File Storage	Cloudflare R2	Production-ready
Process Mgmt	PM2	Configured
Reverse Proxy	Caddy 2 (auto TLS)	Configured
SMS OTP	MSG91	Integrated
Push Notify	Expo Push API	Integrated
Geofencing	PostGIS ST_Buffer + ST_Within	Production-ready

Document prepared by: Gravity Development Team

Last updated: 19 June 2026

Repository: github.com/kamaralam1984/gravitypro